

Project Documentation

Team 38

Team: Abdullah Sheikh, Basel Awwad, Daril Vergesse Sobze Soffack, Emmanuel Bernard
Wona Tschokodeu, Zeynep Cebeci
Teambegleiter: Khush Makhija
Auftraggeber: Frederik Lermyte, Edvaldo Maciel, Jonathan Eisert

Darmstadt



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Bachelorpraktikum
Winter Semester 2025/26
Fachbereich Informatik



Contents

1	Project Description	3
1.1	Vision	3
1.2	Explanation of AI Usage	3
1.3	Implemented Functions	4
1.4	Architecture	4
2	Development Process	5
2.1	Assessment List	5
2.2	Working hours and Planning	6
2.3	Requirements	7
2.4	Quality Assurance Measures	9
3	Conclusion	11

1 Project Description

1.1 Vision

The vision of the project **HYDRA** is to automate the analysis of protein mass spectrometry data in order to support the discovery and characterization of drug candidates. At present, the evaluation of chromatographic MS1 signals is largely performed manually, which limits analysis speed, consistency between experiments, and reproducibility of scientific results.

The software aims to automatically process chromatographic signals, perform Gaussian curve fitting, and calculate hydrodynamic parameters such as the diffusion coefficient (D) and hydrodynamic radius (Rh). In the long term, the project envisions a scalable and user-friendly tool that can integrate advanced statistical or machine-learning methods to gain deeper insights into biophysical and structural biological parameters and thereby accelerate research outcomes.

Domain Description: The project is located in the domain of **biophysical data analysis and automation in online native mass spectrometry**. It focuses on determining hydrodynamic properties of proteins and protein complexes under Taylor dispersion conditions in order to provide insights into molecular size and shape, diffusion behavior, and potential molecular interactions.

Stakeholders: The primary stakeholders are the TU Darmstadt research teams involved in protein biophysics, while the wider scientific community indirectly benefits from the tools automation and reproducibility features.

Technical Concepts: Key parameters include hydrodynamic radius (Rh), diffusion coefficient (D), retention time (tR) and parameters relevant for Taylor dispersion such as (τ) and the Péclet number.

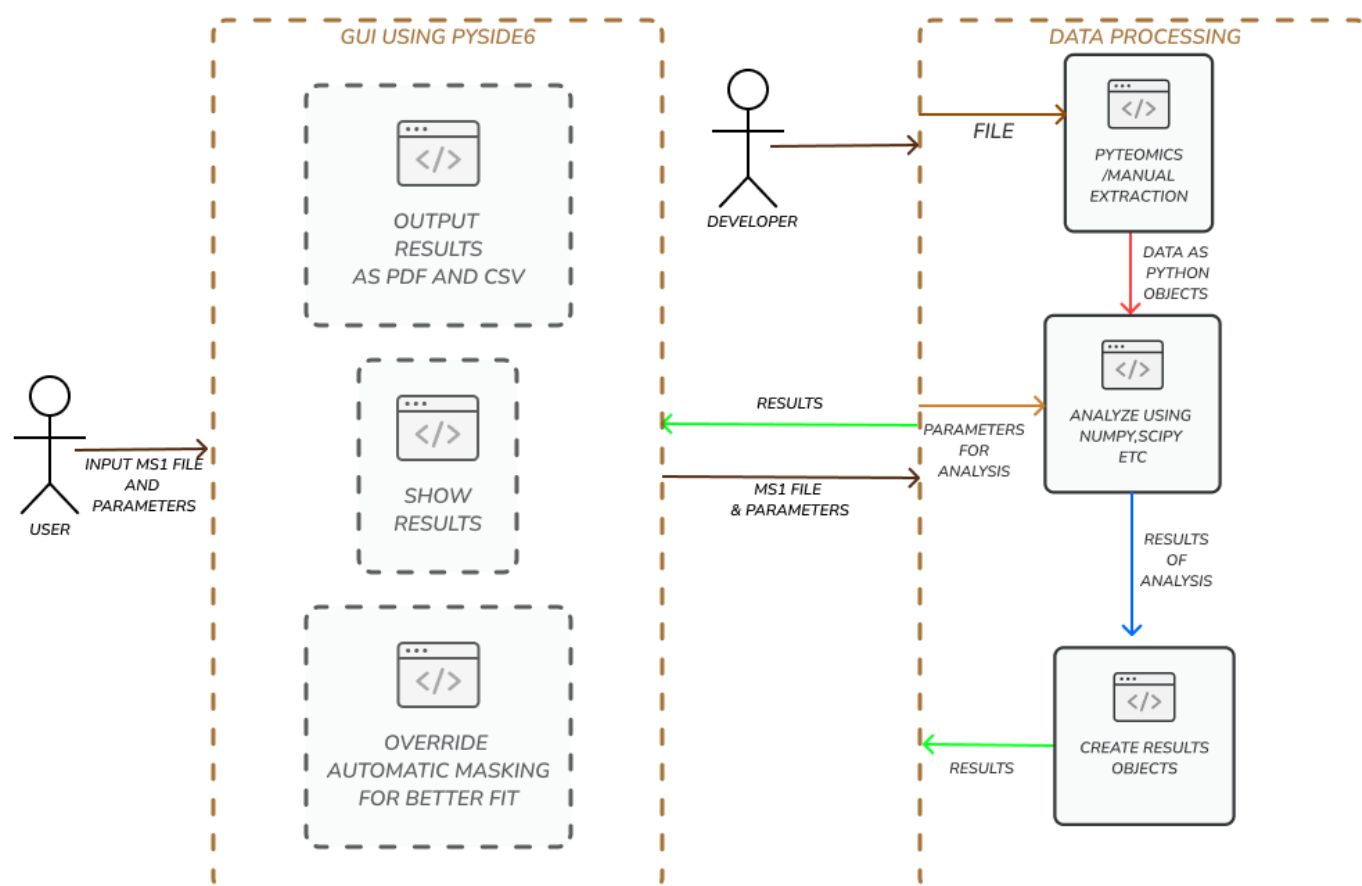
1.2 Explanation of AI Usage

AI-based tools were used exclusively for explanations, guidance or clarifying chemistry terms. All scientific methods and calculations were defined and validated by the customer.

1.3 Implemented Functions

ID	Function
1	Parsing of MS1 files into structured spectra objects containing m/z and intensity arrays
2	Extraction of Extracted Ion Chromatograms (EIC) for configurable m/z windows and charge ranges
3	Support for simultaneous analysis of multiple proteins within one run
4	Gaussian peak fitting with automated quality evaluation using R^2 metric
5	Calculation of biophysical parameters: D , R_h , τ , and P_e
6	Interactive visualization of chromatograms with zoom and navigation tools
7	Result navigation between proteins using Previous/Next controls
8	Input validation, unit handling, and user-oriented error handling
9	Software suggests change in flow-rate if tau or Péclet is outside the Taylor regime.
10	Export of complete analysis reports including plots as PDF documents
11	Allow users to improve the fit, when automatic masking proves inadequate

1.4 Architecture



2 Development Process

This chapter describes the development process that structured our work throughout the semester. We held two regular meetings per week, on Sunday and Wednesday, where we reviewed the progress of the previous week, verified completed tasks, and assessed the reliability and quality of the current software version. Only after confirming that assigned tasks were completed successfully did we move on to new requirements discussed with the customer and distribute them within the team. Our progress was documented continuously to maintain transparency and ensure that no knowledge was lost. Our approach was iterative and based on clearly defined responsibilities. A major challenge occurred during the data analysis phase when results did not meet expectations. By documenting the issue, breaking it into smaller problems, and consulting the customer, we were able to identify the root cause and implement a suitable solution. Quality assurance accompanied the process continuously through systematic testing, reviews and collaborative development practices.

2.1 Assessment List

Project Management

- ☐ Documentation of requirements.
- ☐ Prioritization of tasks after every customer meeting.
- ☐ Tasks allocated to team members according to complexity and estimated time.
- ☐ Biweekly customer meetings to clarify requirements and review progress.
- ☐ Email communication with the customer to resolve questions about scientific background or unclear requirements.
- ☐ Iterative development in two-week cycles.
- ☐ Continuous documentation of progress to maintain transparency and avoid knowledge loss.
- ☐ GitHub for version control.
- ☐ Branching strategy to separate development from main branch.

Team Communication

- ☐ Regular team meetings twice per week (Sunday & Wednesday) to review tasks.
- ☐ Task progress discussed and clarified during team meetings.
- ☐ Use of Discord for informal and immediate communication.

Quality Assurance Measures

- ☐ Edge case testing (empty inputs, unexpected values, boundary conditions).
- ☐ Manual UI testing according to checklists before every customer meeting.
- ☐ Code reviews before merging into the main branch.
- ☐ Pair programming sessions for complex tasks and refactoring.

- ☐ In-code documentation and structured data models for maintainability.
- ☐ Regular updates to the customer to ensure alignment with expectations.

2.2 Working hours and Planning

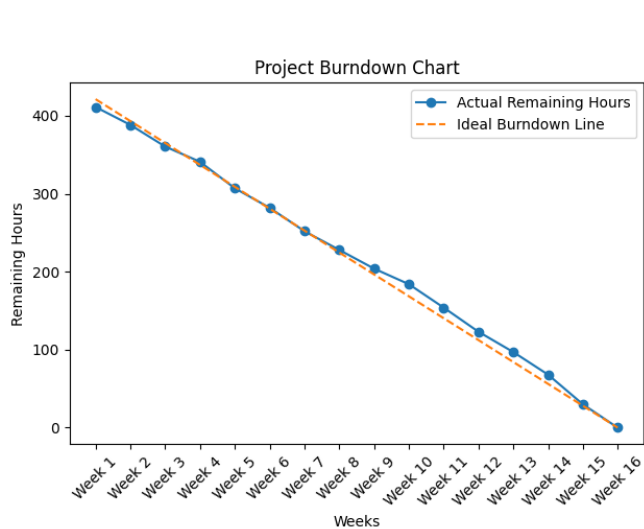


Figure 2.1: Burn-Down Chart

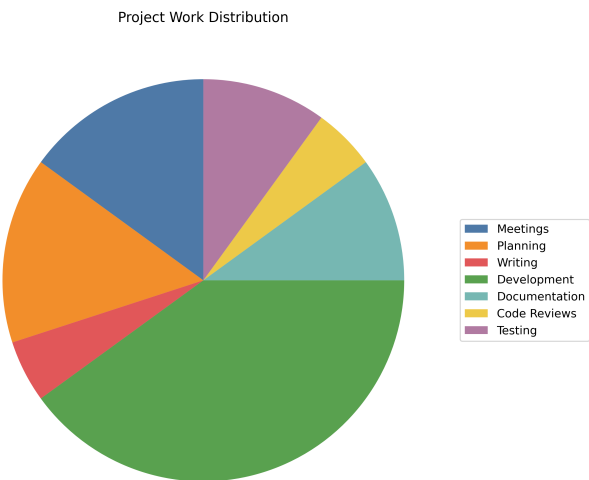


Figure 2.2: Pie Chart

2.3 Requirements

At the beginning of the project, the requirements were defined in close collaboration with the customer. Early on, they provided us with a document explaining relevant concepts and equations required for the calculations, as well as two core requirements that formed the foundation of the project.

As the project progressed, further requirements were gathered through regular biweekly meetings with the customer. During these meetings, new requirements and refinements were documented by the team. After the meeting we would create a to-do list and allocate the tasks to team members. Depending on the scope of a requirement, tasks were either assigned to a single team member or for more complex requirements, worked on collaboratively by two team members.

When uncertainties arose regarding the customer's expectations or the scientific background of a requirement, the team contacted the customer via email to clarify open questions. This direct communication helped prevent misunderstandings during implementation. While the initial document described only the fundamental features of the software, additional requirements emerged naturally as the customer reviewed intermediate results. These new requirements did not fundamentally change the direction of the project but rather extended or refined existing functionality. We also proactively proposed additional features aimed at improving usability, which were then approved by the customer.

The requirements were primarily functional in nature and focused on automating data analysis. A typical requirement involved the calculation of experimentally derived values, such as the hydrodynamic radius (R_h) (see Figure 2.3a), which represented the most time-consuming and technically complex part of the project. Other requirements included the calculation of derived parameters such as tau and the Péclet number based on user input (Figure 2.3b), exporting analysis results as PDF files (Figure 2.3c), accessibility options for color-blindness (Figure 2.3d) and manual data point selection to allow users to refit chromatograms when automatic fitting was inadequate (Figure 2.3e).

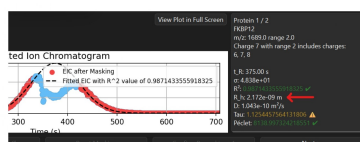


Figure 2.3.a

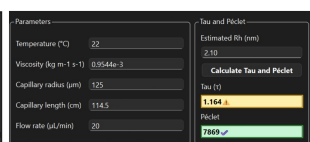


Figure 2.3.b

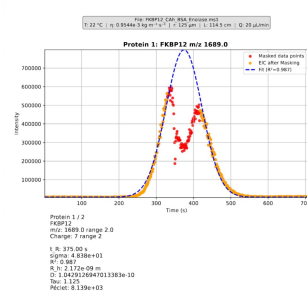


Figure 2.3.c

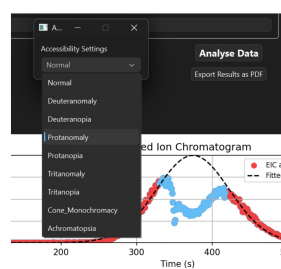


Figure 2.3.d

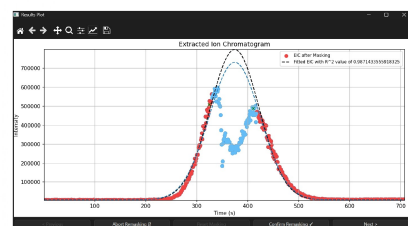


Figure 2.3.e

After the two main requirements defined at the beginning of the project, newly emerging requirements tended to be less time-intensive. These typically involved handling unexpected user input and refining graphical user interface elements. Overall, the time estimates for implementing requirements were mostly accurate. One notable exception was the Gaussian fitting for the extracted ion chromatogram (EIC), which represents the core functionality of the software. (see Figure 2.4a for first attempts at fitting the EIC and 2.4b for the final result.) It was the first thing we started working on and we had planned for it to be finalized before moving on to work on the GUI. However, it became apparent that the fitting algorithm did not perform reliably for all proteins in the provided MS1 data, due to variations in the shape of the EICs. (see Figure 2.5). As a result, manual data point selection had to be subsequently implemented to allow users to improve the fit. This significantly increased the required development time compared to the original estimate. Conversely, the graphical user interface required less time than anticipated; once a rough design was agreed upon, it evolved naturally alongside the implementation of new features.

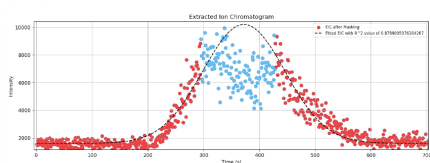


Figure 2.4a

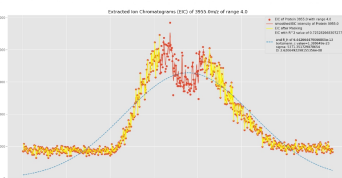


Figure 2.4b

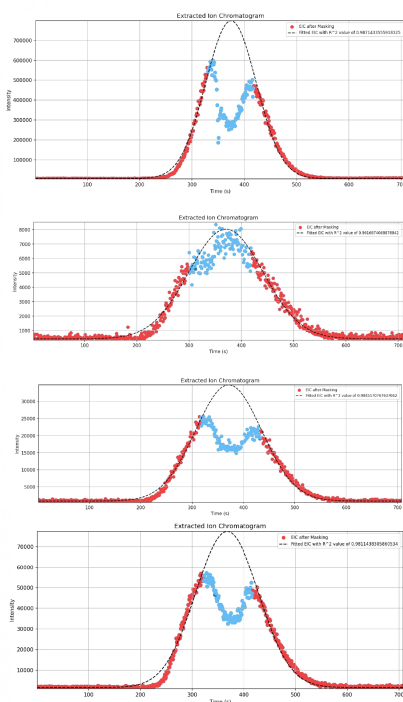


Figure 2.5

The development of the project effectively proceeded in two-week cycles aligned with the biweekly meetings with the customer. Within these intervals, the number of requirements addressed varied depending on their complexity. In some cycles, multiple smaller requirements could be completed, while in others the focus was on a single, more complex feature.

At the end of each meeting with the customer, we prioritized the most important requirements and agreed on the tasks to be completed before the next meeting. We then split the tasks up and everyone had to work on them until our weekly meeting. At our meetings we discussed the progress made and clarified questions that came up, sometimes contacting the customer. We were mostly able to implement the requested functionality within the agreed time frame.

Overall, the requirements management process proved to be useful for the project. Given the size and technical complexity of the software, defining and prioritizing a manageable set of requirements every two weeks helped structure the development process, distribute workload effectively and ensure steady progress.

2.4 Quality Assurance Measures

Throughout the project, we applied several quality assurance measures to ensure stability, correctness, and maintainability of the application. Since the system combines computational logic with a graphical user interface, it was important to continuously verify both the numerical calculations and the interaction between components. Quality assurance was therefore treated as an ongoing process rather than a final step at the end of development.

Systematic Testing Although no formal automated unit testing framework was implemented, we conducted systematic manual testing of all core functionalities. Special attention was given to the mathematical calculations (e.g., Gaussian fitting, hydrodynamic Radius and diffusion coefficient calculation), which were verified using known reference values and plausibility checks.

Edge cases were explicitly tested, including:

- empty or incomplete input fields,
- boundary values for m/z ranges and charge states,
- invalid or inconsistent parameter combinations.

For the graphical user interface, structured scenario-based testing was performed. Typical user workflows were repeatedly executed to ensure that:

- input validation works reliably,
- multiple proteins can be processed sequentially,
- navigation between results functions correctly,
- visualizations and exported reports reflect the computed data consistently.

Whenever a bug was identified, it was first reproduced under controlled conditions, then corrected, and finally re-tested to ensure the issue was fully resolved.

Code Reviews An important quality assurance measure was peer review of code changes. Larger modifications were reviewed by at least one other team member before integration. Reviews focused on logical correctness, readability, modular structure, and adherence to agreed naming conventions. Particular emphasis was placed on clear separation of responsibilities between GUI, controller, and analysis logic.

Code reviews improved overall clarity of the codebase and reduced the likelihood of hidden logical errors.

Pair Programming For more complex features, such as restructuring the architecture or implementing multi-protein support, pair programming was applied. One team member focused on implementation while the other reviewed logic, identified potential edge cases, and discussed architectural decisions in real time. This approach allowed design issues to be detected early and strengthened the shared understanding of the system.

Architectural and Documentation Practices In addition to testing and reviews, we emphasized a modular architecture and clear separation of components. Core analysis logic was kept independent from GUI elements to simplify debugging and improve maintainability. Meaningful naming conventions and in-code documentation were used to enhance readability and long-term sustainability of the project.

Overall, the combination of structured manual testing, peer review, pair programming, and architectural discipline contributed significantly to the robustness and maintainability of the software. Although automated testing was not implemented within the project timeframe, the applied measures ensured a high level of functional reliability.

3 Conclusion

The development of **HYDRA** has been an instructive and challenging journey for us. Reflecting on the past semester, we perceive the project as a success. We not only delivered a functional tool that automates the analysis of protein mass spectrometry data but also established a professional software engineering workflow that we can rely on in future projects.

Reflection on Process and Team

The collaboration within the team was generally positive. A major success factor was our adherence to the Quality Assurance measures described in Section 2.4. Initially, our self-imposed standards for **Manual Tests** and **Code Reviews** felt like a hindrance to our velocity. However, as the complexity of the mathematical models increased, these measures became our safety net. They allowed us to refactor core logic without breaking existing functionality. The use of **Pair Programming** was also a highlight; it was crucial for translating the complex domain knowledge provided by the stakeholders into working code, effectively preventing logic errors that a single developer might have missed.

Obstacles and Learnings

However, the project was not without its hurdles. A significant negative aspect was the steep learning curve regarding the specific biophysical domain (Taylor dispersion and MS1 analysis). We initially underestimated the time required to fully grasp the requirements, which led to some friction in the early development process. Ultimately, this knowledge gap was bridged through independent research coupled with detailed clarifications from the client. Additionally, relying heavily on manual testing for the **GUI** (PySide6) proved to be inefficient; the interface required repetitive manual validation before every milestone, which slowed down our progress. Despite this bottleneck, it did not halt our development, and we persevered to successfully meet our objectives.

Future Improvements

If we were to restart this project, we would make two strategic changes:

- **Earlier Domain Deep-Dive:** We would allocate the entire first sprint solely to understanding the mathematical constraints with the stakeholders to avoid later architectural pivots.
- **UI Automation:** We would implement an automated GUI testing framework (e.g., utilizing Pyside6-compatible test drivers) to reduce the manual testing burden.

In summary, HYDRA has met the stakeholder's vision of a user-friendly, automated analysis tool. For us as a team, the most valuable takeaway is the practical experience of data analysis, and strict quality assurance, which while time-consuming upfront, is the only sustainable way to develop complex scientific software.